耗时优化-特性手册

> **版本：** B012SP1 | **负责人：John Pan** | **最后更新：**
>
> 本手册为"耗时优化"特性的交付文档，涵盖场景描述、收益分析、系统架构、性能指标、使用手册及常见问题。

---

1. 场景描述

1.1 背景

在B008-B010版本，为了提高全流程JOB成功率，合入了很多特性，导致主线单线JOB耗时突破80s，因此在B010版本启动耗时优化特性开发。

1.2 特性定义

耗时优化一阶段为一系列耗时优化特性的集合，包含多个子特性。目标是基于THHB线束场景，优化单条线束JOB耗时，使其逼近15.0s，对齐1.0倍人效。

1.3 适用场景

场景	说明
THHB	耗时优化一阶段是基于THHB线束场景展开的，在其他线束场景下通用性与扩展性有待验证
AIO	AIO场景下的单线JOB流程与THHB基本可以服用，AIO耗时也可参考本项目

---

2. 收益

2.1 效率提升

单条线束JOB耗时从6.29的82s优化至35s内；

注：本文档中写的单条线束JOB均指平均值，即一轮完整的JOB结束后取平均值所需的耗时。

2.2 质量提升

JOB全流程成功率与主线持平

2.3 成本降低

---

3. 系统架构

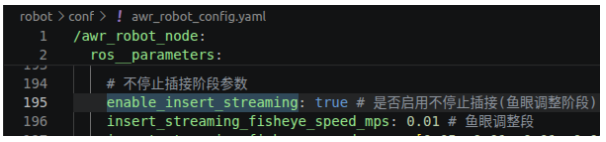
3.1 整体架构

3.1.1 子项目列表



3.1.2 子特性CheckList

特性名	简要描述	测试是否感知	测试注意事项
去结构光	去除结构光轨迹，端子精修完全由内窥镜负责	是	1. 取到料后会直接开始理线，需要注意是否有挂线的风险

上料不停顿	1. 上料两端轨迹合并 2. 插接后提前退出轨迹复播 3. 夹爪控制耗时优化	否	NA
感知优化	1. E2E后处理耗时优 2. perception节点耗时优化	否	NA
WBP在线规划加速&轨迹优化	1. WBP在线规划逻辑优化 2. WBP轨迹加速 3. WBP轨迹优化	是	1. 理线(取料退出位->插接准备位)速度大幅提升，单根线束耗时82s(B010)->45s(B011SP2)； 2. JOB成功率预期不会有大的波动，但是如果发现以下情况请尝试微调点位解决： 2.a 某个理线夹一直过不去； 2.b 频繁挂线； 2.c 频繁挂料盒右上角； 3. 运动速度过快会对XMOTION有潜在影响，可能会遇到JOB过程中机械臂挂死的情况，重新上下电可恢复
轨迹Blending	多段轨迹复播合并，轨迹过度更平滑	否	NA
鱼眼精修不停顿	三段鱼眼轨迹合并为一段，实时更新感知结果	是	1. 本特性只在AIO场景开启，THHB场景默认关闭 2. AIO场景下，可以通过如下配置控制特性开关 
三次插接加速	夹爪控制优化&真同步&减少停顿	否	NA

3.2 核心组件说明

3.2.1 WBP

主控上料退出-->理线段轨迹，负责轨迹优化和加速；

3.2.2 ROBOT

主控插接流程，负责插接控制链路优化；

3.2.3 WBC

执行插接动作，负责跟随性能和交互接口优化；

3.3 关键依赖

组件名	人力需求(人周)
WBP	2.0
ROBOT	3.0
WBC	1.0

---

4. 性能指标

4.1 里程碑

项目对标将在0710发版的B012SP1，预期将主线单根线束的JOB耗时优化至35s；



---

5. 使用手册

5.1 前置条件

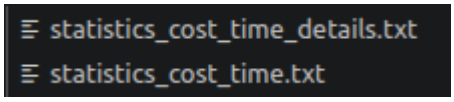
本特性无需开启，更新对应发布版本大包后即可生效。

5.2 验收流程

5.2.1 自主验收

最直接的方法就是通过肉眼观察和录视频的方式记录耗时，推荐通过一轮JOB的总时间倒退单根线束的JOB耗时。

如果需要详细的耗时数据，Thor机器的/apollo路径下会存放两个文件：



statistics_cost_time.txt文件保存了每次JOB的时延打点数据；其中WIRE_CONNECTION和LOAD_WIRE的耗时相加可以大致认为是单个JOB总耗时：

```
JOB_SECTION: 2026-07-04 15:37:51
time_point: LOAD_WIRE, avg = 3975.13 ms, count = 1
time_point: LOAD_WIRE_PREPARATION_POSTURE, avg = 0.183 ms, count = 1
time_point: LOAD_WIRE_PERCEPTION, avg = 3974.36 ms, count = 1
time_point: LOAD_WIRE_POSE_DETECTION, avg = 451.265 ms, count = 2
time_point: WIRE_CONNECTION, avg = 52782.4 ms, count = 1
time_point: CONNECTION_BOTH_REPLAY, avg = 52774.2 ms, count = 1
time_point: CONNECTION_BOTH_REPLAY_PHOTO_POS, avg = 18313.9 ms, count = 1
time_point: CONNECTION_BOTH_REPLAY_INSERT_BOTH, avg = 27622.6 ms, count = 1
time_point: CONNECTION_BOTH_REPLAY_LOAD_WIRE_PRE_POS, avg = 6837.32 ms, count = 1
time_point: PERCEPTION_INSERT, avg = 0.4402 ms, count = 10
time_point: PERCEPTION_LOAD, avg = 0.084 ms, count = 19
time_point: LOAD_WIRE_PERCEPTION_POLL, avg = 451.262 ms, count = 2
time_point: LOAD_WIRE_GRASP_MOVE, avg = 1713.43 ms, count = 1
time_point: LOAD_WIRE_GRASP_ACTION, avg = 306.811 ms, count = 1
time_point: LOAD_WIRE_EXIT, avg = 1050.62 ms, count = 1
time_point: INSERT_SEND_PERCEPTION_LEFT, avg = 0.7876 ms, count = 5
time_point: INSERT_SEND_PERCEPTION_RIGHT, avg = 0.7142 ms, count = 5
time_point: INSERT_POLL_LEFT, avg = 703.923 ms, count = 10
time_point: INSERT_CAL_POSE_LEFT, avg = 0.464667 ms, count = 3
time_point: INSERT_CAL_POSE_RIGHT, avg = 0.345 ms, count = 3
time_point: INSERT_GRIPPER, avg = 268.51 ms, count = 8
time_point: INSERT_CAL_INSERT_POSE, avg = 0.293 ms, count = 1
time_point: INSERT_CAL_EXIT_POSE, avg = 0.633 ms, count = 1
time_point: INSERT_MOVE_RPC, avg = 0.769286 ms, count = 14
time_point: INSERT_MOVE_WAIT, avg = 2327.96 ms, count = 5
time_point: INSERT_FAST_MOVE_WAIT, avg = 1098.95 ms, count = 8
time_point: INSERT_EXIT_WAIT, avg = 1302.88 ms, count = 1
time_point: INSERT_MOVE_WAIT_VELOCITY, avg = 1466.56 ms, count = 16
time_point: INSERT_MOVE_WAIT_POSE, avg = 0.007625 ms, count = 16
time_point: GOAL_TRAJ_PLAN, avg = 3294.83 ms, count = 2
time_point: GOAL_FOLLOW_JOINTS, avg = 3581.84 ms, count = 2
time_point: GOAL_GRIPPER, avg = 245.067 ms, count = 10
```

注意这里记录的JOB时间是结束时间

5.2.2 推荐验收工具

可以从http://red-gitlab.tars-ai.com/theodor/scripts下载耗时分析工具。通过如下方式运行：

```
tars@tars-System-Product-Name:~/log$ ./analysis_latency.sh Latency/133/0703 17:49:26
extracted section: input lines 6017-6260 (244 lines)
wrote 27 records -> /home/tars/log/Latency/133/0703/workflow_174926.md
tars@tars-System-Product-Name:~/log$ ./analysis_latency.sh Latency/133/0704
input: /home/tars/log/Latency/133/0704/statistics_cost_time.txt
rules config: /home/tars/proj/scripts/latency_analysis/conf/latency_exception_rules.yaml
job config: /home/tars/proj/scripts/latency_analysis/conf/job_config.yaml
total JOB_SECTION: 36
valid JOB_SECTION: 30
invalid JOB_SECTION: 6
wrote: /home/tars/log/Latency/133/0704/statistics_latency.md
tars@tars-System-Product-Name:~/log$ ./analysis_latency.sh Latency/133/0704 17:58:41
```

Latency/133/0704是存放statistics_cost_time.txt文件的路径。运行脚本后目录下会自动生成一个statistics_latency.md文件，里面有详细的耗时分析数据。

JOB运行情况

JOB通过率统计

状态	数量	占比	平均总时延(s)	平均上料时延(s)	平均理线时延(s)	平均插接时延(s)
PASS	11	100.00%	32.060	3.324	8.255	20.481
FAIL	0	0.00%	NA	NA	NA	NA

JOB运行情况一览

JOB时间	JOB运行时间(s)	状态	原因
2026-07-09 22:10:44	30.340	PASS	-
2026-07-09 22:11:17	30.459	PASS	-
2026-07-09 22:11:45	25.588	PASS	-
2026-07-09 22:12:16	29.252	PASS	-
2026-07-09 22:12:44	25.476	PASS	-
2026-07-09 22:13:14	28.035	PASS	-
2026-07-09 22:13:44	27.636	PASS	-
2026-07-09 22:14:13	26.805	PASS	-
2026-07-09 22:14:45	29.144	PASS	-
2026-07-09 22:15:13	25.472	PASS	-
2026-07-09 22:15:40	25.053	PASS	-

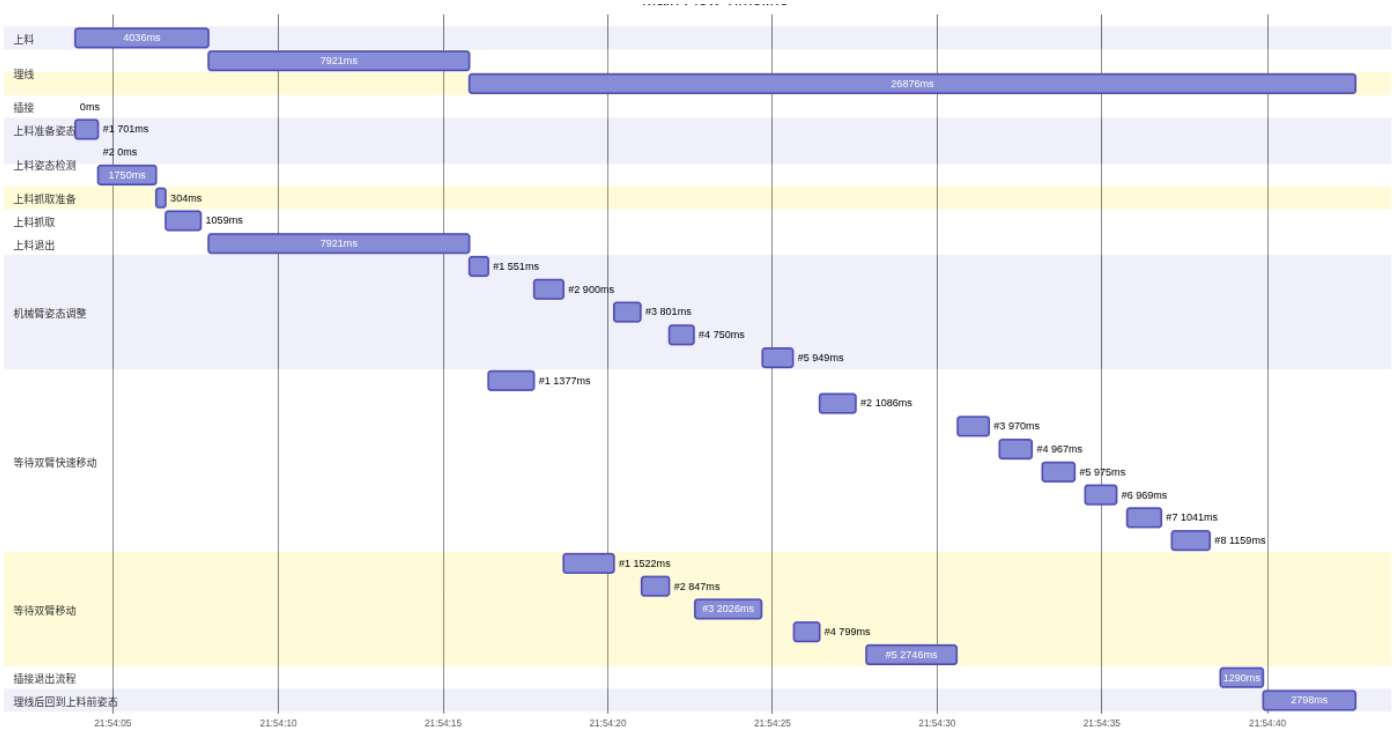
5.3 可视化界面

使用5.2.2中提供的分析工具，运行如下命令：

```
0704/statistics_latency.md
g$ ./analysis_latency.sh Latency/133/0704 17:58:41
```

在对应目录下可以找到如下文件，其中生成了单根线束的细分耗时：





6. 常见问题

> **文档维护：** 如有问题或建议，请联系XXXX